# A Causal Learning Framework for Enhancing Robustness of Source Code Models

JUNYAO YE, Huazhong University of Science and Technology, China
ZHEN LI*, Huazhong University of Science and Technology, China
XI TANG, Huazhong University of Science and Technology, China
DEQING ZOU, Huazhong University of Science and Technology, China
SHOUHUAI XU, University of Colorado Colorado Springs, USA
WEIZHONG QIANG, Huazhong University of Science and Technology, China
HAI JIN, Huazhong University of Science and Technology, China

*Deep Learning* (DL) models are useful for many software engineering tasks. However, these models are susceptible to adversarial attacks, partly because they learn *spurious features* that incur *spurious correlations* between these features and model predictions. In this paper, we tackle the problem with a novel *causal learning* framework, dubbed CausalCode, which leverages causal inference principles to mitigate spurious correlations. At a high level, CausalCode can be characterized as follows: (i) it uses *causal data augmentation* to generate *intervention examples* to disrupt spurious correlations; (ii) it leverages *regularization* to learn *invariant representations* that prefer causal features to spurious features; (iii) it can enhance the robustness of multiple DL models for source code-based software engineering tasks because it is task-agnostic and model-agnostic. To evaluate its effectiveness, we conduct comprehensive experiments on two models (i.e., CodeBERT and GraphCodeBERT), with respect to four software engineering tasks (i.e., defect detection, functionality classification, code translation, and code repair). Experimental results show that CausalCode outperforms the state-of-the-art approaches in enhancing the robustness of these models.

CCS Concepts: • **Software and its engineering → Software development techniques**.

Additional Key Words and Phrases: Causal Learning, Adversarial Robustness, Code Models

**ACM Reference Format:**
Junyao Ye, Zhen Li, Xi Tang, Deqing Zou, Shouhuai Xu, Weizhong Qiang, and Hai Jin. 2025. A Causal Learning Framework for Enhancing Robustness of Source Code Models. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE117 (July 2025), 24 pages. https://doi.org/10.1145/3729387

*Corresponding author.

Authors' Contact Information: Junyao Ye, Zhen Li, Xi Tang, Deqing Zou: National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, 430074, China and JinYinHu Laboratory, Wuhan 430040, China; e-mails: d202280655@hust.edu.cn, zh_li@hust.edu.cn, m202472229@hust.edu.cn, deqingzou@hust.edu.cn; Weizhong Qiang: School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, 430074, China and JinYinHu Laboratory, Wuhan 430040, China; e-mail: wzqiang@hust.edu.cn; Shouhuai Xu: Department of Computer Science, University of Colorado Colorado Springs, Colorado Springs, CO 80918, USA; e-mail: sxu@uccs.edu; Hai Jin: National Engineering Research Center for Big Data Technology and System, Services Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, 430074, China; e-mail: hjin@hust.edu.cn.


ACM 2994-970X/2025/7-ARTFSE117
https://doi.org/10.1145/3729387

## 1 Introduction

*Deep Learning* (DL) models have been widely used in many software engineering tasks, such as code defect detection, code function identification, code refinement, and code translation [17, 18, 26, 32, 41, 43]. However, studies show that adversarial attacks can easily deceive or evade these DL models [10, 14, 51–53], by reducing their accuracy from over 90% to 10% or less [14, 46, 51]. This susceptibility of DL models to adversarial attacks stems from the following fundamental limitation of DL models in learning from code: Rather than capturing the true *causal relationships* between the semantic properties that determine program behavior and model predictions, they often learn the *spurious correlations* between superficial syntactic patterns and model predictions, namely statistically significant but causally irrelevant associations. For instance, in software vulnerability detection, models might incorrectly associate specific variable naming patterns with vulnerabilities, rather than capturing unsafe API usage patterns or improper error handling that causally determine code vulnerability. As a consequence, these models can be easily evaded by simply manipulating the values of spurious features without affecting the code's semantics or functionality.

The robustness problem mentioned above has attracted a fair amount of attention. In general, there are two approaches: (i) *adversarial training*, which incorporates adversarial examples into the training data [7, 15, 16, 37, 38, 46, 47, 52], and (ii) *contrastive learning*, which employs an unsupervised contrastive pre-training step [9, 19]. However, these approaches have three limitations: (i) they extend the distributions of training data rather than eliminating spurious correlations; (ii) their perturbation strategies lack theoretical foundations that can be leveraged to distinguish between causal features and spurious features; and (iii) they have demonstrated limited effectiveness (e.g., [51] shows that attacks can still achieve over 70% success rate against DL classifiers that are fortified by adversarial training). This calls for new approaches to make DL models robust against adversarial attacks. In principle, causal learning can address the preceding three limitations because they exhibit the following two unique properties: (i) it can leverage program semantics, as reflected by Abstract Syntax Trees (ASTs) and Control Flow Graphs (CFGs), to capture relationships between code elements and identify causal features; and (ii) it can leverage the hierarchical structure of code to naturally distinguish semantic features (e.g., control flow and data dependencies) from stylistic features (e.g., naming and formatting), which allows us to use semantic equivalence transformations to perform precise causal interventions.

**Our contributions.** The limited success of these approaches motivates us to explore a novel perspective: leveraging causal learning to enhance the robustness of DL models in software engineering tasks. While causal learning has shown promise in other domains such as image recognition and natural language processing [2, 22, 33, 50], its application to software engineering presents unique challenges and opportunities.

In this paper, we make two contributions. First, we propose a new approach to enhancing the robustness of DL-based software engineering models, based on the notion of *causality* [27]. Specifically, we propose a causal learning framework, dubbed CausalCode, to enhance the robustness of DL models for source code tasks. At a high level, our approach can cope with the problem of *spurious* features, which is suffered by the state-of-the-art approaches (i.e., adversarial training and contrastive learning). While the idea may sound simple, we encounter two technical challenges.

- The first challenge is to elucidate and leverage the relationships between features and DL model predictions (e.g., labels) in software engineering tasks. These tasks encounter a unique challenge imposed by program semantics and structure, which has no counterpart in the other settings mentioned above (e.g., image recognition). To address this challenge, we leverage causal learning to learn *causal* features, which have a direct influence on DL model predictions and thus can enhance their robustness, while recalling that *spurious* features can be exploited by attackers to

craft adversarial examples to evade DL models. Specifically, we propose a novel *data augmentation* method to eliminate spurious features. This method differs from adversarial training because it leverages the *do-operator* in causal analysis to generate *intervention examples*, which can eliminate spurious correlations between code samples and prediction results.

- The second challenge is to reduce the impact of spurious features while amplifying the significance of causal features in source code models. By leveraging the idea of *d-separation* in causal analysis, we show that when a model uniformly represents all the intervention examples associated with a given example (in the original training set), the model learns invariant representations by disregarding spurious features and accentuates causal features. This leads to a regularization technique that minimizes the representational distance between original examples and their intervention counterparts. This strategy not only enhances the performance of code models but also improves their robustness against perturbation and adversarial attacks.

Second, we evaluate the effectiveness of CausalCode by conducting experiments on two widely investigated source code classification tasks, *function classification* and *defect detection*, as well as two code generation tasks, *code repair* and *code translation*. These experiments primarily use two extensively employed Pre-trained Language Models (PrLMs) for code: CodeBERT [4] and GraphCodeBERT [6], with additional experiments demonstrating feasibility on StarCoder [12]. These models are state-of-the-art DL models for understanding and processing source code. Experimental results show that CausalCode significantly improves the robustness and performance of these models, while outperforming the existing approaches to enhancing robustness (i.e., *adversarial learning* and *contrastive learning*). Moreover, we investigate the impact of three elements on the model's performance and robustness: intervention examples generation, regularization strategy, and dataset size. We demonstrate that these elements collectively enhance model performance and reduce the attack success rate. We have made the source code of CausalCode publicly available at https://github.com/CGCL-codes/CausalCode.

**Paper outline.** Section 2 reviews preliminary knowledge. Section 3 explores the application of causal learning to code models. Section 4 describes the CausalCode framework in detail. Section 5 presents our experimental results. Section 6 reviews related prior studies. Section 7 discusses the limitations of the present study. Section 8 concludes the paper.

## 2 Preliminaries

**Causal graph.** The notion of *causal graph* [27] is introduced to represent the causal relationships between variables. Specifically, causal graphs are Directed Acyclic Graphs (DAGs) for analyzing how variables may affect each other via the directionality of influence (rather than mere association). Nodes in a causal graph represent variables, while arcs (i.e., directed edges) represent hypothesized causal influences between variables [27]. For instance, Fig. 1(a) presents a causal graph illustrating the relationship between variable $X$, which represents code complexity, and variable $Y$, which represents bug occurrence, namely that $X$ causally influences $Y$.

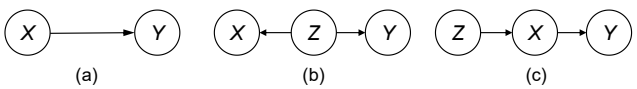



Fig. 1. Illustration of (a) simple causal relationship, (b) confounding relationship where $Z$ affects both $X$ and $Y$, and (c) chain relationship where $Z \perp\!\!\!\perp Y \mid X$.

**Structural equations.** A causal graph is typically accompanied by a set of *structural equations* that mathematically specify how each variable is determined by its direct causes (i.e., parents in the graph) [27]. Each structural equation takes the form of $V = f_V(PA_V, U_V)$, where $V$ is a variable in the causal graph, $f_V$ is a function, $PA_V$ represents its parent variables, and $U_V$ represents

unobserved factors. Intuitively, a structural equation defines a deterministic functional relationship that specifies how a variable is generated from its parent variables and exogenous disturbances. For instance, in Fig. 1(a), bug occurrence $Y$ can be described by a structural equation $Y = f_Y(X, U_Y)$, where $X$ is code complexity and $U_Y$ represents other unobserved factors affecting bug occurrence (e.g., test coverage, developer experience, and code review quality); in this case, the structural equation captures how bug occurrence is determined by code complexity and other unobserved factors. These equations pave a mathematical foundation for conducting intervention operations, as elaborated below.

**Intervention and do-operator.** The notion of *intervention* [27] is introduced for untangling or differentiating causal relationships from correlations. Specifically, an intervention is to set a variable to a specific value and investigate the consequential impact on other variables. For instance, corresponding to Fig. 1(a), an intervention to set the value of variable $X$ as "low" or "high" and then observe the effect on the downstream variable $Y$ and inspect the causal influence of $X$ on $Y$. Formally, the notion of intervention is described by the *do-operator*, denoted by $\text{do}(variable = value)$, which describes that the *variable* is set to the *value*. This naturally leads to the notion of *intervention probability* $P(Y = y \mid \text{do}(X = x))$, which quantifies the probability of $Y = y$ when $X$ takes the value $x$; this permits the isolation of $X$'s effect on $Y$ from other possible confounding influences. Note that the notion of intervention probability is different from the notion of *conditional probability* $P(Y = y \mid X = x)$. The difference can be understood as follows [27]: intervention probability $P(Y = y \mid \text{do}(X = x))$ measures the effect of actively setting $X = x$ through external manipulation, while conditional probability $P(Y = y \mid X = x)$ reflects the passive observation of $X = x$ in its natural state. This distinction is crucial in confounding scenarios, which can be illustrated using Fig. 1(b), where $Z$ represents an unobserved confounder that affects both $X$ and $Y$. In such cases, conditional probability $P(Y = y \mid X = x)$ may not accurately represent the causal effect of $X$ on $Y$ due to the confounding influence of $Z$; by contrast, intervention probability $P(Y = y \mid \text{do}(X = x))$ can isolate the true causal relationship between $X$ and $Y$ by blocking the backdoor path from $Z$.

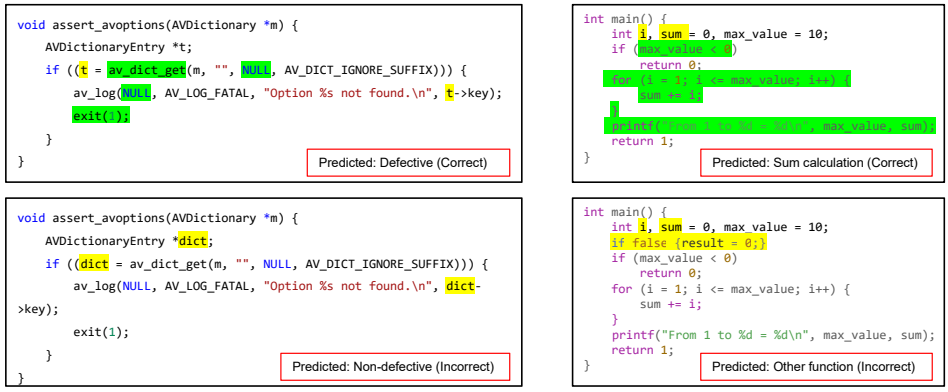
**D-separation.** The notion of *d-separation* [27] provides a mechanism for describing whether one variable is conditionally independent of another, by detecting whether a path starting from one variable and ending at another variable is "blocked" by variables. Fig. 1(c) illustrates a chain structure $Z \rightarrow X \rightarrow Y$, where $Z$ influences $Y$ through $X$. When $X$ is fixed (either by conditioning or intervention), the path from $Z$ to $Y$ is "blocked", making $Z$ and $Y$ conditionally independent of each other; this is denoted by $Z \perp\!\!\!\perp Y \mid X$. Suppose $Z$ represents code quality, $X$ represents code complexity, and $Y$ represents bug occurrence. Code quality $Z$ affects bug occurrence $Y$ through its impact on code complexity $X$. When we condition on code complexity $X$, code quality $Z$ provides no additional information about bug occurrence $Y$, demonstrating how conditioning on $X$ blocks the information flow from $Z$ to $Y$.

## 3 Causal Learning for Robust Code Models

In this section, we explore the idea of leveraging causal learning to enhance the robustness of code models against perturbation and adversarial attacks, especially in eliminating spurious features through data augmentation and learning invariant representations.

### 3.1 Causal vs. Spurious Features in Machine Learning-Based Code Models

In the application of machine learning to code analysis, distinguishing between causal and spurious features is critical for developing robust models. For instance, in machine learning-based software defect detection, causal features include semantic patterns, such as unsafe API usage (e.g., `av_dict_get` with `NULL`), control flow structures for error handling (e.g., `exit(1)`), and data flow relationships reflecting improper input validation; Fig. 2(a) illustrates these causal features. On




**(a) Defect detection: variable name changed from "t" to "dict"** **(b) Functional classification: Insert "dead code"**

Fig. 2. Illustration of spurious features (yellow) vs. causal features (green) in code analysis tasks. (a) In the context of *defect detection*, the function `assert_avoptions` checks for invalid dictionary options. The defect lies in potential NULL pointer dereference when using `av_dict_get`, which could lead to undefined behavior if the key is not found. The causal features include the unsafe API call pattern and error handling with `exit(1)`. The model correctly predicts the original code with variable name `t` as "Defective", but incorrectly predicts the semantically identical code with variable name `dict` as "Non-defective". (b) In the context of *function classification*, the code implements a sum calculation algorithm. The causal features include the loop structure and accumulation pattern (`sum += i`). The model correctly classifies the original code as "Sum calculation", but when dead code (`if false {result = 0;}`) is inserted, the model incorrectly classifies it as "Other function" despite identical functionality.

the other hand, spurious features include variable names, code formatting, and non-functional elements that can be modified without affecting the program's semantics or functionalities.

In software functionality classification, causal features encompass algorithmic structures and core operations. Fig. 2(b) illustrates causal features in function classification. The original code implements a sum calculation function that computes the sum of integers from 1 to a maximum value. The causal features include the loop structure, accumulation pattern (`sum += i`), and output formatting. The model correctly classifies this as a sum calculation function. However, when syntactically valid but semantically irrelevant dead code (`if false {result = 0;}`) is inserted, the model misclassifies the function as belonging to a different category, despite the code's actual functionality remaining identical, again showing reliance on spurious features. These spurious features are frequently targeted in adversarial attacks against various code analysis tasks [30, 46, 51]. It is important to note that causal features are task-specific [28]. For example, the `printf` function may serve as a causal feature for functionality classification but could be spurious for other tasks.

### 3.2 Structural Causal Model for Code Analysis

To apply causal learning to code tasks, we begin with the notion of Structural Causal Model (SCM) [27]. As illustrated in Fig. 3(a), a SCM is a Directed Acyclic Graph (DAG) that represents variables as nodes and relationships as arcs, where dashed arcs represent *correlations* and solid arcs represent *causal* relationships. Consider a deep learning model that predicts class $Y$ based on an input $X$ by utilizing a set of causal features $X_S$ and a set of spurious features $X_F$. Fig. 3(a) illustrates how $S$ (semantics) and $F$ (stylistic) influence these features.

Specifically, this study addresses the task of code functionality classification, where $Y$ represents code functionality and $X$ represents an arbitrary piece of source code (text). Then, $X$ can be represented by two kinds of observable and directly modifiable variables: (i) the *semantics* variable, denoted by $S$, which represents code functionality; and (ii) the *stylistic* variable, denoted by $F$,

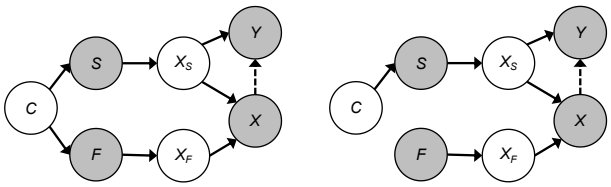



Fig. 3. (a) A causal graph, where gray nodes represent observed variables, dashed arrows represent correlations, and solid arrows represent causal relationships. (b) The same causal graph after making intervention on $F$.

which represents stylistic elements such as variable names and dead code. Both $S$ and $F$ can be directly observed in the code and manipulated by humans. The node $C$ in Fig. 3(a) represents a hidden confounder, which is a latent variable not directly observed. For example, $C$ could denote programmer expertise or the development environment, influencing both stylistic elements ($F$) and semantic elements ($S$). The SCM shown in Fig. 3(a) corresponds to the following structural equations [27]:

$$\begin{aligned} &S := g_S(C, \varepsilon_S), \quad F := g_F(C, \varepsilon_F), \qquad X_S := g_{X_S}(S, \varepsilon_{X_S}), \quad X_F := g_{X_F}(F, \varepsilon_{X_F}), \\ &X := g_X(X_F, X_S, \varepsilon_X), \quad Y := g_Y(X_S, \varepsilon_Y) \end{aligned} \tag{1}$$

where the structural functions $g_S$, $g_F$, $g_{X_S}$, $g_{X_F}$, $g_X$, and $g_Y$ capture the relationships among the variables. The terms $\varepsilon_S, \varepsilon_F, \varepsilon_{X_S}, \varepsilon_{X_F}, \varepsilon_X, \varepsilon_Y$ represent mutually independent noise terms. These noise terms account for measurement errors, unobserved influencing factors, or inherent randomness in the model, ensuring that the SCM captures the stochastic nature of real-world phenomena [28]. In this context, $X_S$ represents features derived from semantic elements $S$ (e.g., abstract syntax tree of a for-loop), while $X_F$ represents features derived from stylistic elements $F$ (e.g., embeddings of variable names).

In Fig. 3, the hidden confounder node $C$ introduces a *backdoor path* [27] $F \leftarrow C \rightarrow S$, which allows stylistic variable $F$ to influence semantics variable $S$ indirectly, resulting in a non-causal association. Consequently, $F$ and $S$ are not independent of each other, which is denoted by $p(F, S) \neq p(F)p(S)$. When predicting the class $Y$ of $X$, machine learning models often rely on both high-level features $X_F$ and $X_S$. This reliance on correlated features that do not have a direct causal relationship with the target variable is known as a *spurious correlation*. Spurious correlations can lead to models that perform well on training data but fail to generalize to new situations or when faced with adversarial examples. To address the correlation between $F$ and $S$, we propose conducting an intervention on $F$ to render $F$ and $S$ independent, namely $p(S|do(F)) = p(S)$. As shown in Fig. 3(b), the intervention on $F$ removes the arc from hidden confounder $C$ to $F$, effectively eliminating the backdoor path $F \leftarrow C \rightarrow S$. In practice, this intervention could involve randomly changing variable names ($F$) while keeping the code's functionality ($S$) constant. This intervention strategy aligns with the independent causal mechanisms principle [35], where the semantic ($S$) and stylistic ($F$) mechanisms are modular and independently manipulable.

Let us illustrate this concept using a concrete software engineering example from defect detection, as shown in Fig. 2(a). Consider function `assert_avoptions`, which checks for invalid dictionary options. The causal features have direct causal relationships with potential defects. The semantic variable $S$ captures program behaviors via the AST and CFG. Meanwhile, the stylistic variable $F$ includes the variable name choice (i.e., `t` vs. `dict`), which has no causal impact on program correctness. The SCM framework helps understand how these variables interact. For instance, when a hidden confounder $C$ (e.g., developer experience) influences both semantic choices (e.g., error handling patterns) and stylistic choices (e.g., variable naming conventions), it creates a backdoor path $F \leftarrow C \rightarrow S$. This explains why models might incorrectly associate certain variable naming

patterns with defect likelihood, leading to spurious correlations, which makes a model fail under adversarial attacks.

### 3.3 Eliminating Spurious Correlations Through Causal Intervention

It is known that *data augmentation* can simulate interventional data for intervening on $F$ [8]. For this purpose, we propose a novel data augmentation method grounded in causal analysis. This method not only simulates interventions but also aids code models in learning invariant representations, thereby further mitigating the impact of spurious features on model robustness.

**Causal data augmentation for enhancing robustness.** From a causal perspective, it is known that adversarial distributions exploit the differences in conditional probability distributions between labels and spurious features in the presence vs. absence of adversarial attacks, and that reducing these differences may enhance model robustness [54]. As shown in Fig. 3(b), we can represent the data augmentation process as an intervention on a given SCM, namely intervention on $F$ and denoted by $do(F)$. This intervention, following Pearl's do-calculus [27], severs the causal link from the hidden confounder $C$ to $F$, resulting in a modified SCM:

$$\begin{aligned} &S := g_S(C, \varepsilon_S), \quad F := f, \quad \text{where } f \sim P(F) \quad X_S := g_{X_S}(S, \varepsilon_{X_S}), \quad X_F := g_{X_F}(F, \varepsilon_{X_F}), \\ &X := g_X(X_F, X_S, \varepsilon_X), \quad Y := g_Y(X_S, \varepsilon_Y) \end{aligned} \tag{2}$$

In this modified SCM, $F$ is no longer influenced by $C$ but is instead sampled from its marginal distribution $P(F)$. This intervention breaks the spurious association between $F$ and $S$ because now we have $P(S|do(F)) = P(S)$. By using the *do-operator* to impose intervention on $F$ (i.e., by manipulating the values of spurious features without altering the code semantics, which can be achieved via semantics-preserving code transformations), we can generate augmented data. Each augmented example $x^{aug}$ individually can be seen as a counterfactual instance. We observe that the two known data augmentation approaches, namely random sampling and adversarial example generation [48, 51], cannot adequately reduce the difference incurred by natural vs. adversarial distributions. This is illustrated in Fig. 4(b), where adversarial distributions are visualized as the stars in the red region because adversarial attacks often follow the direction of gradient descent to lower a model's effectiveness [23, 49, 51]. As shown in Fig. 4(a), the blue and red crosses represent potential transformations of an example, forming a randomly sampled set. On the other hand, Fig. 4(b) demonstrates that adversarial examples may only partially cover the space of possible examples, as indicated by the green trajectory. This limitation arises because adversarial attack strategies often terminate upon successfully deceiving the model.

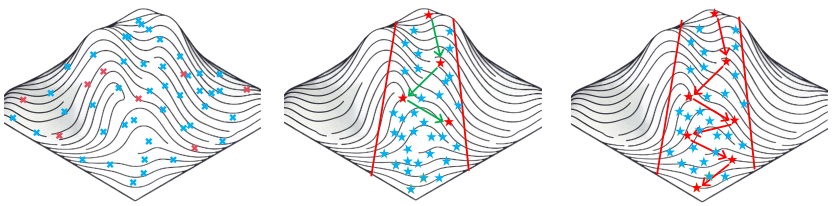



Fig. 4. Illustration of three strategies for generating intervention examples, where blue crosses and stars represent all potential transformations of a given example, while red crosses and stars represent the selection logic. (a) Random sampling from potential transformations. (b) Adversarial method and CausalCode encompass the entire red boundary region (denoted by stars) as a candidate set for gradient descent. In the red region, the direction of the fastest gradient descent is indicated by arrows, where the green arrow represents the strategy for generating adversarial examples. (c) The red arrow represents CausalCode.

To approximate the distribution of adversarial examples, we propose a method for generating intervention examples, which is highlighted in Fig. 4(c). This method, inspired by the *Projected Gradient Descent* (PGD) technique in image processing [23], iteratively updates an intervention example in the direction of gradient descent. Unlike image processing where continuous perturbations are feasible, code transformations are discrete modifications without manipulating the semantic validity of a program. Our approach, as illustrated in Fig. 4(c), employs a multi-path intervention process. For each iteration $t$, we generate and select candidates according to:

$$\mathcal{S}_t = \bigcup_{j=1}^{m} \text{Transform}(x_t, r_j), \quad \mathbf{x}_{t+1} \sim \text{Uniform}(\text{Top}_k(\mathcal{S}_t)) \tag{3}$$

where $m$ is the size of the replacement candidate set $\mathcal{R}$, and $\text{Top}_k(\cdot)$ selects the $k$ candidates that best align with the gradient descent direction to reduce distribution discrepancy. The subsequent uniform sampling from these top candidates prevents local optimum entrapment, while maintaining the gradient descent trajectory. This design generates a Markov chain of interventions that simultaneously preserves semantic validity and ensures comprehensive exploration of the perturbation space. Details of these methods will be presented in Section 4.2.

**Learning invariant representations.** To further mitigate the influence of spurious features, we propose learning an Invariant Representation $\Phi(x)$ that is independent of $F$. This assures that regardless of how adversarial attacks may alter examples, classifier $h(\Phi(X))$ will consistently make the same prediction $Y$ as long as the semantic variable $S$ remains unchanged. Formally, we seek to achieve $P(\Phi(X)|do(F = f)) = P(\Phi(X))$, meaning that the distribution of the learned representation is invariant despite interventions on $F$. For this purpose, we leverage the concept of d-separation in the context of causal inference [27] to identify causal and spurious features, by analyzing the conditional independence relationships in a given SCM. Learning invariant representations requires that $X_S$ satisfies the invariance condition $X_S \perp\!\!\!\perp F \mid S$. To ensure this, we propose making the feature representations $\Phi(x)$ of original examples $\mathbf{x}_i$ and their corresponding intervention examples $\mathbf{x}_i^{aug}$, which share the same $X_S$, have a distance of zero in the representation space. This approach approximates the theoretical invariance condition while remaining computationally feasible [24]. Details of these methods will be presented in Section 4.3.

## 4 The CausalCode Framework

### 4.1 Overview

As illustrated in Fig. 5, CausalCode takes a model $M$ and a training set $D_1$ as input, and outputs an enhanced model $M^+$, which exhibits enhanced robustness against adversarial attacks compared to $M$. This transformation is achieved through two key steps: (i) expanding the training data using causal intervention, and (ii) learning invariant representations from a dataset comprising both the original training set $D_1$ and the intervention examples from $D_1$. To make CausalCode effective, we need to address two core technical questions:

- How does CausalCode generate intervention examples? This will be addressed in Step 1 of CausalCode (Section 4.2).
- How does CausalCode train a robust model capable of learning an invariant representation $\Phi$? This will be addressed in Step 2 of CausalCode (Section 4.3).

### 4.2 Step 1: Expanding Training Data

In Section 3.3, we have explained the derivation of our causal data augmentation method. In this section, we describe how to expand training data using this augmentation method. Algorithm 1 and Fig. 4(c) shows how to generate intervention examples from a given training set $D_1$ =

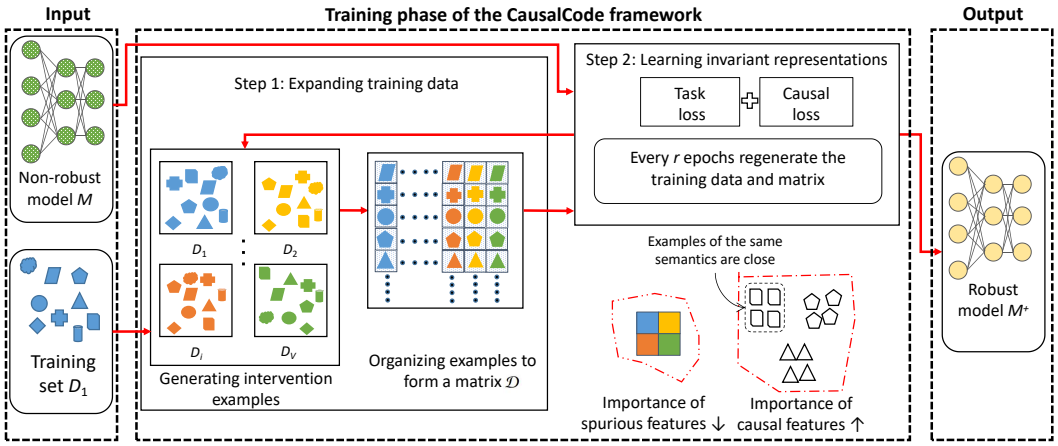



Fig. 5. Overview of CausalCode, which expands the training dataset (Step 1) and learns invariant representations via causal features (Step 2). In the matrix, examples possessing the same causal features are represented by the same shape, while colors represent types of spurious features. At a high level, CausalCode elevates the importance of causal features and makes examples with the same functionality close to each other in the learned representation space, while reducing the importance of spurious features.

**Algorithm 1** Generating intervention examples

1: **Input:** Non-robust model $M$ learned from training set $D_1 = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, with the maximum number of iterations $iter_{max}$ and replacement candidate set $\mathcal{R} = \{r_1, ..., r_m\}$.
2: **Output:** Augmented training set $D_{new}$.
3: **for** each $\mathbf{x}_i$ in $D_1$ **do**
4: $iter_{stop} \leftarrow$ random$(1, iter_{max})$; {generating random number of iterations}
5: $I \leftarrow$ Scan$(\mathbf{x}_i)$; {identifying potential intervention points}
6: Initialize intervention example $\mathbf{x}_i^{new} \leftarrow \mathbf{x}_i$ and iteration count $iter_{count} \leftarrow 0$;
7: **while** $iter_{count} < iter_{stop}$ **do**
8: **for** each $m_j$ in $I$ **do**
9: $\mathcal{S} \leftarrow$ GenerateExamples$(\mathbf{x}_i^{new}, m_j, \mathcal{R})$; {Generate examples using all replacement candidate sets $\mathcal{R}$ for changes at intervention points $m_j$}
10: $\mathbf{cf}_{topk} \leftarrow$ SelectCandidates$(\mathcal{S}, \mathbf{x}_i, M, k)$; {Choose the top $k$ examples that most closely match the gradient descent direction of the model}
11: $\mathbf{x}_i^{new} \leftarrow$ randomSelect$(\mathbf{cf}_{topk})$;{Randomly select one of the top $k$ candidate examples as the input for the next iteration}
12: $iter_{count} \leftarrow iter_{count} + 1$;
13: **end for**
14: **end while**
15: **end for**
16: $D_{new} \leftarrow \{\mathbf{x}_1^{new}, \mathbf{x}_2^{new}, \ldots, \mathbf{x}_n^{new}\}$;
17: **return** $D_{new}$;

$\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n\}$, a maximum number of iterations $iter_{max}$, a replacement candidate set $\mathcal{R}$ which comprises $m$ elements, and a non-robust model $M$ that is to be enhanced. The output is a new training set of intervention examples $D_{new}$. The algorithm iterates on each example $\mathbf{x}_i \in D_1$ $(1 \leq i \leq n)$. Initially, a random stopping point $iter_{stop}$ is determined, corresponding to the number of arrows in Fig. 4 (c), between 1 and $iter_{max}$ (Line 4), to provide coverage of all possible interventions. For each example $\mathbf{x}_i$, the algorithm identifies potential intervention points $I$ where iterative transformations can take place without altering the semantics of the code (Line 5). With $\mathbf{x}_i^{new}$ initialized as the original example and an iteration count set to zero, the algorithm enters a loop during which it

applies interventions to the candidate set $\mathcal{R}$ at $I$ to generate transformed examples $\mathcal{S}$ (Lines 7-14). Within each iteration, the top $k$ examples that most closely align with the model's gradient descent direction are selected (Line 10), where alignment is based on the similarity score between the embedding of an example and that of an intervention example, namely

$$SC(\mathbf{x}_i^{s_j}, \mathbf{x}_i) = \frac{v(\mathbf{x}_i^{s_j}) - v(\mathbf{x}_i)}{\|v(\mathbf{x}_i^{s_j}) - v(\mathbf{x}_i)\|_2} \frac{\partial L(y, M(\mathbf{x}_i))}{\partial v(\mathbf{x}_i)} \tag{4}$$

where $v(\mathbf{x}_i^{s_j})$ and $v(\mathbf{x}_i)$ are respectively the embedding vectors of $\mathbf{x}_i^{s_j}$ (i.e., one example in $\mathcal{S}$) and $\mathbf{x}_i$, $M(\mathbf{x}_i)$ is the model's prediction for $\mathbf{x}_i$, and $\frac{\partial L(y, M(\mathbf{x}_i))}{\partial v(\mathbf{x}_i)}$ is the embedding-level gradient vector of $\mathbf{x}_i$. Instead of computing gradients for each candidate, the algorithm calculates the gradient only once for the original identifier. Thus the overall overhead is comparable to generating adversarial examples. Given these top examples, a random one is selected as the new intervention example, representing a random selection from candidates in the gradient descent direction (red area) in Fig. 4(c). The algorithm halts when each example in $D_1$ is processed. Finally, the algorithm outputs intervention examples to form an expanded training set $D_{new}$ (Line 17).

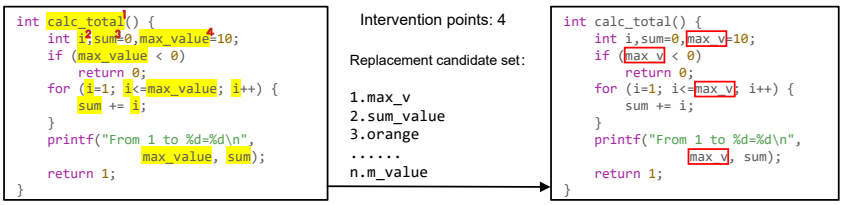



Fig. 6. Illustration of intervention, where yellow color highlights the locations where interventions are made while preserving program/code semantics (i.e., functionality), the middle part provides a candidate set of points for conducting interventions, and red boxes highlight the interventions that are made.

Fig. 6 illustrates Algorithm 1 with respect to intervening on identifiers as spurious features, using a concrete code instance. The intervention points are locations where modifications do not alter the code's semantics. The intervention method considers all variable and function names as potential intervention points. The candidate set $\mathcal{R}$ encompasses feasible interventions at each intervention point. For identifier transformations, the set comprises all strings from the dataset that adhere to variable naming conventions, with their sizes capped (set to 5000 in our experiments) to prevent uncontrolled expansion. The aforementioned process describes the expansion of a single training set. We can trivially extend this to multiple datasets, by extending $D_1$ to $\mathcal{D} = \{D_1, D_2, \ldots, D_V\}$ for some $V$, where we often encounter $|D_1| = |D_2| = \ldots = |D_V| = n$. The goal is to generate $V - 1$ intervention examples for each example $\mathbf{x}_i^{(1)} \in D_1$ where $1 \leq i \leq n$, resulting in $\mathbf{x}_i^{(2)}, \ldots, \mathbf{x}_i^{(V)}$. These $V - 1$ intervention examples form the sets $D_2, \ldots, D_V$, where each $D_j = \{\mathbf{x}_1^{(j)}, \mathbf{x}_2^{(j)}, \ldots, \mathbf{x}_n^{(j)}\}$ for $2 \leq j \leq V$. Finally, the following matrix of examples obtained by sampling examples from $\mathcal{D}$ is used as the input for model training in Step 2:

$$\mathcal{D} = \begin{pmatrix} \mathbf{x}_1^{(1)} & \mathbf{x}_1^{(2)} & \ldots & \mathbf{x}_1^{(k)} \\ \mathbf{x}_2^{(1)} & \mathbf{x}_2^{(2)} & \ldots & \mathbf{x}_2^{(k)} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_n^{(1)} & \mathbf{x}_n^{(2)} & \ldots & \mathbf{x}_n^{(k)} \end{pmatrix} \tag{5}$$

where each original example $\mathbf{x}_i^{(1)}$ $(1 \leq i \leq n)$ is aligned with its $k - 1$ intervention examples in the same row, indicating data augmentation. Unlike contrastive learning, which typically involves

organizing and processing positive and negative example pairs, CausalCode pairs examples that share the same causal features to facilitate learning invariant representations.

### 4.3 Step 2: Learning Invariant Representations

Having expanded our training data through causal data augmentation in Step 1, we now focus on learning invariant representations that are robust to spurious features. As discussed in Section 3.3, our goal is to learn a feature representation $\Phi(x)$ that is independent of the fragility features $F$. To achieve this, we enforce the condition that the feature representations $\Phi(x)$ of the original examples $\mathbf{x}_i$ and their corresponding intervention examples $\mathbf{x'}_i$, which share the same semantic features $X_S$, should have zero distance in the representation space. This can be formalized as:

$$\sum_{j_1 \neq j_2} \texttt{dist}\left(\Phi\left(\mathbf{x}_i^{(j_1)}\right), \Phi\left(\mathbf{x}_i^{(j_2)}\right)\right) = 0 \tag{6}$$

where $\mathbf{x}_i^{(j_1)} \in D_{j_1}$ and $\mathbf{x}_i^{(j_2)} \in D_{j_2}$ $(j_1 \neq j_2)$ represent pairs of examples sharing the same program semantics, $\Phi$ is the feature extraction function, and `dist` is a distance metric (e.g., the $l_2$ distance).

Moreover, we introduce $\mathcal{L}_{CausalCode}$ as a *regularizer* to aid the minimization of the disparity between the original examples and their intervention examples, namely

$$\mathcal{L}_{Causal} = \sum_{1 \leq i \leq n; 1 \leq j_1, j_2 \leq v; j_1 \neq j_2} \texttt{dist}\left(\Phi\left(\mathbf{x}_i^{(j_1)}\right), \Phi\left(\mathbf{x}_i^{(j_2)}\right)\right) \tag{7}$$

where minimizing $\mathcal{L}_{Causal}$ encourages the model to learn causal features that capture the semantics of the code. However, solely minimizing $\mathcal{L}_{Causal}$ may not be sufficient because it would result in $\mathcal{L}_{Causal} = 0$ when dealing with null code. To address this issue, we introduce an additional condition to ensure that $X_S$ effectively conveys information about the semantics and $Y$. To achieve this, we incorporate the following standard cross-entropy loss function $\mathcal{L}_{Task}$:

$$\mathcal{L}_{Task} = \sum_{1 <= i <= n; 1 \leq j \leq v;} l\left(G\left(\Phi\left(\mathbf{x}_i^{(j)}\right)\right), y_i^{(j)}\right) \tag{8}$$

where $l$ is an appropriate loss function (e.g., cross-entropy loss), $G$ is the model, $\Phi$ is the feature extractor, $\mathbf{x}_i^{(j)}$ is the feature vector of the $i$th example, $y_i^{(j)}$ is the label of the $i$th example, and $n$ is the total number of examples in the dataset.

To optimize the overall objective, we combine $\mathcal{L}_{Causal}$ and $\mathcal{L}_{Task}$, leading to the loss function $\mathcal{L}_{CausalCode}$:

$$\mathcal{L}_{CausalCode} = \lambda \mathcal{L}_{Causal} + \mathcal{L}_{Task} \tag{9}$$

where $\lambda$ serves as a hyperparameter. The appropriate value of $\lambda$ is determined according to the code task and the model in question. By minimizing $\mathcal{L}_{CausalCode}$ during the learning process, CausalCode achieves the objective highlighted in Fig. 5, ensuring that representations of examples with the same semantics are close to each other in the representation space.

Algorithm 2 outlines the training process of the CausalCode-enabled model. The algorithm begins with $\lambda = 0$ (Line 3), $N_s$ with a small value (Line 5), and the non-robust model $M$ for initial learning of data features. The training begins by minimizing the cross-entropy loss (Line 6). The datasets $\mathcal{D}$, consisting of the original training set and $v - 1$ datasets of intervention examples, are used in batches (Line 9). To prioritize causal features $X_S$ and diminish the influence of spurious features $X_F$, as the CausalCode loss decreases, the algorithm gradually increases the ratio of the causally invariant loss until it reaches 1, thereby minimizing the CausalCode loss described in Eq.(9) (Lines 8-17). The expanded training set $\mathcal{D}$ is updated every $r$ epochs (Line 19). The algorithm terminates when the loss converges (Line 4). The periodic update of the randomly perturbed training set

**Algorithm 2** Training a CausalCode-enabled model

**Input:** Non-robust model $M$ trained on dataset $D_1$, number of augmented datasets $V$, epochs per update $r$, starting epoch $N_s$ for $\mathcal{L}_{\text{CausalCode}}$ calculation
**Output:** CausalCode-enabled model $M^+$.
Initialize $\lambda \leftarrow 0$;
**while** training not converged **do**
  **if** $epoch \leq N_s$ **then**
    Minimize $\mathcal{L}_{\text{Task}}$ as in Eq. (8);
  **end if**
  **if** $epoch > N_s$ **then**
    **for** $batch \sim \mathcal{D}$ **do**
      Minimize $\mathcal{L}_{\text{CausalCode}} = \lambda \mathcal{L}_{\text{Causal}} + \mathcal{L}_{\text{Task}}$ as in Eq. (9);
    **end for**
  **end if**
  **if** $\lambda < 1$ **and** $\mathcal{L}_{\text{CausalCode}}$ decreases **then**
    $\lambda \leftarrow \lambda + 0.1$;
  **else**
    $\lambda \leftarrow \lambda - 0.1$; {Decrease $\lambda$ if $\mathcal{L}_{\text{CausalCode}}$ does not decrease}
  **end if**
  **if** epoch mod $r = 0$ **then**
    Update $\mathcal{D}$ {Except for dataset $D_1$}
  **end if**
  epoch $\leftarrow$ epoch + 1
**end while**
**return** $M^+$;

enables the model to learn diverse code variations, extract essential features, and acquire invariant representations, leading to a robust model $M^+$.

## 5 Experiments and Results

Our experiments target four *Research Questions* (RQs):

- **RQ1**: Can CausalCode enhance robustness against adversarial attacks?
- **RQ2**: Can CausalCode learn causal features?
- **RQ3**: Which elements of CausalCode contribute to improving a model's robustness?
- **RQ4**: Can CausalCode be extended to addressing code generation tasks?

### 5.1 Experimental Setup

**Datasets.** We consider four datasets that are widely used and publicly available. Each example in these datasets ensures semantic correctness, a crucial requirement for conducting semantics-preserving transformations.

- **POJ-104** [26]. This dataset comprises 51,976 programs categorized into 104 distinct functionalities. It is tailored for the task of function classification, aiding in the classification of programs according to their functionalities.
- **CodeChef** [51]. This dataset is introduced for defect detection and includes 33,822 programs categorized into four categories: "OK" (code without defects, passing all test cases), "WA" (incorrect output for test cases), "TLE" (execution timeout), and "RE" (runtime error).
- **CodeTrans** [21]. This dataset includes parallel functions between Java and C#, comprising a total of 11,800 paired functions. It is designed for code conversion between programming languages, with an average sequence length of 38.51 tokens for Java functions and 46.16 for C# functions.

- **Bugs2fixs** [39]. This dataset comprises 65,454 Java functions, consisting of parallel pairs of buggy and corrected code. The objective is to transform erroneous code into its correct version.

**Models.** For each source code-based task, we consider the following two deep learning models.

- **CodeBERT** [4]. It is a pre-trained model based on transformer architecture, intended to process Natural Language and Programming Language (NL-PL) pairs as input. It is a multi-programming-lingual model pre-trained on NL-PL pairs in 6 programming languages (Python, Java, JavaScript, PHP, Ruby, and Go). CodeBERT has achieved state-of-the-art performance in various downstream tasks, including natural language code search, NL-PL Probing, and code documentation generation.
- **GraphCodeBERT** [6]. It is a pre-trained model designed to take a graph representation of code as input, where nodes represent variables and edges represent data flows. It is chosen for its success in achieving state-of-the-art results across various code-related downstream tasks.
- **StarCoder2** [20]. It is a decoder-only model available in 3B, 7B, and 15B parameter variants, pre-trained on the Stack v2 dataset covering 619 programming languages. For classification tasks, we adapt it by adding a classification head on top of the last hidden state.

**Attacks.** The robustness of these models has been evaluated against random perturbation attacks and state-of-the-art adversarial attacks. We limited the maximum number of attack attempts per sample to 40 for both types of attacks, following the approach of Zhang et al. [51]. The adversarial attacks employed in our experiments include the Metropolis-Hastings sampling-based identifier renaming attack (MHM) [52], the attack utilizing gradient information for source code manipulation (CARROT) [51]. CARROT encompasses two subtypes: manipulation of identifiers (C-Identifier) and insertion of dead code (C-DeadCode). Additionally, an approach employing black-box attacks by identifying vulnerable tokens (CodeAttack) [10]. These attacks are used in our experiments. It is important to note that both CARROT and MHM are adversarial attacks for code classification tasks, while CodeAttack focuses on adversarial attacks for code generation tasks.

**Model training.** For the classification task, we divide the dataset into training, testing, and validation sets following the same ratios (64%: 20%: 16%) as used by CARROT [51]. For generation tasks, we adhere to the configurations established by CodeAttack [10]. Specifically, for the code translation task, we utilize a total of 11,800 paired functions, allocating 1,000 for testing and the remaining for training and validation. For the code repair task, we employ a smaller dataset comprising 46,680 training samples, 5,835 for validation, and 5,835 for testing. To mitigate potential biases, we repeat the adversarial attack on a 20% uniformly sampled subset of the test set five times and report the average results.

**Evaluation metrics.** For code classification tasks, we use two metrics to evaluate the classification capability of a model $G$, such as $M$ or $M^+$. One metric is *Accuracy*, denoted by Acc, which represents the ratio of test examples, that are correctly classified by model $G$. We use $\Delta_{drop}$ to represent the change in Accuracy before and after the model is attacked, namely $\Delta_{drop} = Acc_{before} - Acc_{after}$. Note that a larger $\Delta_{drop}$ indicates lower robustness. The other metric is *attack success rate* metric, denoted by ASR, which indicates the percentage of misclassified adversarial examples generated from a given set of examples. Specifically, let $X$ denote a set of examples that are correctly classified by model $G$, meaning that $G(x)$ returns the ground-truth label for $x \in X$. Let $x'$ be an adversarial example generated from $x \in X$. Then, we have ASR $= \frac{|\{x|x \in X \wedge G(x') \neq G(x)\}|}{|X|}$. The higher the ASR, the more effective the attack.

For code generation tasks, we use CodeBLEU [31] to gauge the quality of the generated code relative to the ground truth. CodeBLEU provides a multifaceted evaluation of code quality by assessing the syntactic correctness, semantic accuracy, and data-flow correctness of generated

code snippets. Following CodeAttack [10], we employ the $\Delta_{drop}$ metric to measure the difference between a model's performance before and after waging adversarial attacks, namely $\Delta_{drop} = \text{CodeBLEU}_{\text{before}} - \text{CodeBLEU}_{\text{after}}$.

**Experimental configurations.** In terms of code classification, we use the Adam optimizer with batch size 64, a maximum number of iterations $iter_{max}$ 30, and weight decay $3e-5$. Both kinds of classifiers are trained for 28 epochs with an early stopping criterion being set to 3 epochs. In terms of code generation, for a *code translation* task, we configure both the maximum source length and the target length as 512 tokens, and use a beam search with size 10, and $iter_{max}$ 10; for a *code refinement* task, we set the maximum source length and the target length to be 256 tokens with a reduced beam size 5, and $iter_{max}$ 20. Both tasks are trained for a maximum of 36 epochs with early stopping after 3 epochs and a learning rate of $5e-5$. For RQ1, RQ2, and RQ4, we augment the training set with one additional training set and the starting epoch number $N_s$ for CausalCode loss calculation being set to 3. For StarCoder2-3B, we employ mixed precision training with bfloat16, using a batch size of 64 and a learning rate of 3e-4 with linear warmup over 10% steps. More information about the parameters is described in our open-source repository mentioned above. We conduct experiments on a server with an Intel(R) Xeon(R) Gold 6226R CPU at 2.90GHz and NVIDIA RTX A6000 GPUs with 48 GB memory, running Ubuntu 22.04.

**Selecting other robustness-enhancement methods for comparison purposes**. For comparison purposes, we consider four baseline defense methods. The first two methods belong to the adversarial training approach, namely CARROT-T [51] and ALERT-T [46], which are selected (from a number of candidate methods including [7, 14, 52]) because of their effectiveness across various code tasks and models. The third method belongs to a contrastive learning approach. While both ContraCode [9] and ContraBERT [19] leverage contrastive learning to conduct unsupervised training to enhance the robustness of pre-trained models, we note that ContraCode is pre-trained and only experimented with the JavaScript language and that ContraBERT is trained on a diverse set of languages and experimented with adversarial attacks. Thus, we select ContraBERT. We summarize the defense methods as follows. The fourth method represents the first application of causal learning in software engineering models, which is closely related to our research.

- **CARROT-T** [51]: It iteratively generates and integrates small batches of adversarial examples into the training set for training a non-robust model $M$.
- **ALERT-T** [46]: It conducts a single round of adversarial perturbation across the entire training set, while retaining both the adversarial examples that successfully evade the model and the unsuccessful adversarial examples that maximally reduce the model's confidence in the correct predictions.
- **ContraBERT** [19]: It introduces nine data augmentation operations on programming language and natural language datasets to create variant examples. The resulting augmented datasets are then used to enhance the robustness of pre-trained models via contrastive learning with CodeBERT and GraphCodeBERT.
- **CausalVul** [30]: It introduces causality into deep learning vulnerability detection by leveraging the do-calculus-based causal learning algorithms to eliminate spurious features. This method enhances model accuracy and robustness across various state-of-the-art models and datasets.

### 5.2 Performance and Robustness against Advanced Attacks (RQ1)

**Accuracy improvement and robustness against perturbation attacks on classification tasks**. Table 1 summarizes the accuracy and Attack Success Rate (ASR) of CodeBERT and GraphCodeBERT on two classification tasks under random perturbation attacks. Function classification classifiers demonstrate superior performance, achieving approximately 19% higher accuracy and a 24.93%

lower ASR compared to defect detection classifiers. This discrepancy may be attributed to the nature of the datasets: the POJ-104 dataset contains 104 types of functionally identical code, while the CodeChef dataset encompasses limited defect types with high semantic variety. For the same code task, these models exhibit similar effectiveness. On average, the five methods achieve a modest 0.50% increase in accuracy and an 8.05% decrease in ASR for function classification. For defect detection, they yield a 1.12% accuracy increase and a more substantial 16.06% ASR reduction. Notably, CausalCode demonstrates the most significant improvements, with an accuracy increase of 1.25% and an ASR reduction of 31.95% compared to the baseline. CausalVul performs slightly weaker than CausalCode, while ContraBERT achieves an average accuracy increase of 0.76% while robustness decreases. Between the two adversarial learning strategies, ALERT-T outperforms CARROT-T, achieving a 1.16% increase in accuracy and demonstrating superior ASR reduction.

Table 1. Random Perturbation (RP) results on classification tasks for CodeBERT and GraphCodeBERT models, enhanced with five different methods (metrics unit: %)

| Model | Function Classification | | | | | | Defect Detection | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | RP-Identifier | | | RP-DeadCode | | | RP-Identifier | | | RP-DeadCode | | |
| | Acc | $\Delta_{drop}$ | ASR | Acc | $\Delta_{drop}$ | ASR | Acc | $\Delta_{drop}$ | ASR | Acc | $\Delta_{drop}$ | ASR |
| CodeBERT | 96.70 | 34.53 | 35.71 ± 0.24 | 96.70 | 22.47 | 19.78 ± 0.18 | 77.48 | 49.86 | 58.13 ± 0.21 | 77.48 | 45.51 | 50.95 ± 0.16 |
| +ContraBERT | 97.17 | 55.50 | 57.12 ± 0.12 | 97.04 | 26.04 | 23.38 ± 0.19 | 78.60 | 52.62 | 63.58 ± 0.18 | 78.60 | 41.60 | 47.59 ± 0.25 |
| +CARROT-T | **97.56** | 41.14 | 42.17 ± 0.25 | 96.88 | 31.45 | 25.50 ± 0.20 | 77.47 | 49.85 | 58.13 ± 0.17 | 77.48 | 47.05 | 52.99 ± 0.24 |
| +ALERT-T | 97.50 | 14.82 | 15.20 ± 0.16 | 97.46 | 18.40 | 10.69 ± 0.15 | **78.77** | 47.69 | 54.51 ± 0.28 | 79.87 | 32.70 | 34.88 ± 0.23 |
| +CausalVul | 97.23 | 11.61 | 9.70 ± 0.19 | 97.23 | 14.00 | 12.23 ± 0.15 | 78.34 | 23.10 | 26.20 ± 0.20 | 78.34 | 54.77 | 68.58 ± 0.28 |
| +CausalCode | 97.24 | **10.70** | **7.32 ± 0.15** | **97.09** | **9.69** | **3.92 ± 0.15** | 78.08 | **18.53** | **16.72 ± 0.18** | **80.30** | **16.87** | **15.35 ± 0.21** |
| GraphCodeBERT | 97.03 | 36.12 | 37.23 ± 0.27 | 97.03 | 14.66 | 13.78 ± 0.17 | 78.05 | 58.60 | 73.91 ± 0.21 | 78.05 | 40.95 | 49.93 ± 0.17 |
| +ContraBERT | 97.49 | 39.17 | 40.18 ± 0.26 | 97.49 | 19.39 | 18.22 ± 0.18 | 79.12 | 45.67 | 55.43 ± 0.27 | 79.12 | 42.39 | 50.72 ± 0.20 |
| +CARROT-T | 97.44 | 38.07 | 39.07 ± 0.16 | 97.47 | 21.51 | 19.67 ± 0.18 | 79.44 | 55.67 | 68.66 ± 0.16 | 78.05 | 40.58 | 49.41 ± 0.18 |
| +ALERT-T | 97.45 | 18.14 | 18.61 ± 0.18 | 97.50 | 6.44 | 5.01 ± 0.23 | 79.63 | 43.71 | 45.63 ± 0.23 | 79.60 | 16.20 | 20.95 ± 0.25 |
| +CausalVul | 97.50 | 16.08 | 8.48 ± 0.21 | 97.50 | 6.09 | 5.77 ± 0.15 | 78.08 | 36.83 | 43.71 ± 0.23 | 78.08 | 33.43 | 39.07 ± 0.20 |
| +CausalCode | **97.51** | **12.14** | **6.99 ± 0.18** | **97.56** | **3.76** | **2.29 ± 0.25** | **80.21** | **21.55** | **21.17 ± 0.23** | **80.53** | **8.30** | **10.08 ± 0.17** |

Table 2. Adversarial attack results on classification tasks for CodeBERT and GraphCodeBERT models, enhanced with five different methods (metrics unit: %)

| Model | Function Classification | | | | | | Defect Detection | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | MHM | | CARROT-I | | CARROT-D | | MHM | | CARROT-I | | CARROT-D | |
| | $\Delta_{drop}$ | ASR | $\Delta_{drop}$ | ASR | $\Delta_{drop}$ | ASR | $\Delta_{drop}$ | ASR | $\Delta_{drop}$ | ASR | $\Delta_{drop}$ | ASR |
| CodeBERT | 11.64 | 12.04 | 69.59 | 71.97 | 33.63 | 27.20 | 61.28 | 61.65 | 61.28 | 79.09 | 54.25 | 64.35 |
| +ContraBERT | 12.79 | 13.16 | 78.01 | 80.28 | 26.94 | 24.35 | 61.65 | 59.29 | 61.65 | 78.43 | 49.93 | 59.40 |
| +CARROT-T | 11.75 | 12.04 | 63.31 | 64.89 | 32.18 | 25.69 | 60.98 | 60.26 | 60.98 | 78.72 | 55.25 | 66.43 |
| +ALERT-T | 7.52 | 7.71 | 29.58 | 30.34 | 21.96 | 14.75 | 58.30 | 63.20 | 58.30 | 74.01 | 41.57 | 47.08 |
| +CausalVul | 6.52 | 69.76 | 28.58 | 29.34 | 16.33 | 12.58 | 57.30 | 84.88 | 34.17 | 38.47 | 61.10 | 76.00 |
| +CausalCode | **5.34** | **5.49** | **19.27** | **19.82** | **10.79** | **5.68** | **26.36** | **30.25** | **26.36** | **33.76** | **20.47** | **21.03** |
| GraphCodeBERT | 12.97 | 13.37 | 89.33 | 92.06 | 21.63 | 19.84 | 71.57 | 62.06 | 71.57 | 91.70 | 52.38 | 65.36 |
| +ContraBERT | 10.97 | 11.25 | 64.68 | 66.35 | 16.69 | 15.39 | 62.02 | 48.48 | 62.02 | 78.39 | 52.39 | 64.13 |
| +CARROT-T | 13.70 | 14.06 | 89.59 | 91.94 | 39.39 | 38.54 | 72.79 | 61.32 | 72.79 | 91.63 | 52.35 | 65.30 |
| +ALERT-T | 8.08 | 8.29 | 29.71 | 30.49 | 7.37 | 5.92 | 62.59 | 47.98 | 62.59 | 78.60 | 24.30 | 29.52 |
| +CausalVul | 7.08 | 64.86 | 28.71 | 29.49 | 10.10 | 9.62 | 61.59 | 46.77 | **29.44** | **32.89** | 38.07 | 48.89 |
| +CausalCode | **4.84** | **4.96** | **16.06** | **16.47** | **4.76** | **3.10** | **37.45** | **37.48** | 37.45 | 46.69 | **11.43** | **12.49** |

**Experimental results with adversarial attacks on classification tasks**. Table 2 summarizes the experimental results of adversarial attacks. For function classification, we observe the following: (i) Three types of attacks exhibit an average ASR of 39.41% on the original model and 35.13% on the contrastive learning pre-trained model ContraBERT. Against advanced adversarial attacks, the enhancement offered by ContraBERT is significantly limited. (ii) Among two adversarial learning enhancement methods, ALERT-T, which involves retraining with a large volume of adversarial samples, shows superior effectiveness over CARROT-T, which continues training with a smaller batch of adversarial samples. (iii) Models enhanced with CausalCode significantly outperform state-of-the-art methods, reducing the ASR by 30.16%. This improvement establishes CausalCode as

Table 3. Random Perturbation(RP) and Adversarial attacks results on defect detection tasks for Starcoder2, enhanced with five different methods (metrics unit: %)

| Model | RP-Identifier | | | RP-DeadCode | | | MHM | | | CARROT-I | | | CARROT-D | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Acc | $\Delta_{drop}$ | ASR | Acc | $\Delta_{drop}$ | ASR | Acc | $\Delta_{drop}$ | ASR | Acc | $\Delta_{drop}$ | ASR | Acc | $\Delta_{drop}$ | ASR |
| Starcoder2 | 77.60 | 45.70 | 58.81 | 77.60 | 49.59 | 62.94 | 77.60 | 50.77 | 65.43 | 77.60 | 73.80 | 95.23 | 77.60 | 59.07 | 75.66 |
| +CARROT-T | 77.64 | 44.51 | 57.23 | 77.59 | 43.29 | 55.54 | 77.96 | 48.69 | 62.37 | 77.63 | 62.60 | 80.67 | 77.39 | 53.82 | 69.57 |
| +ALERT-T | **79.02** | 41.99 | 53.13 | 78.52 | 29.34 | 37.32 | 78.85 | 46.98 | 59.61 | **79.15** | 60.45 | 76.37 | 78.67 | 38.17 | 48.55 |
| +CausalVul | 78.28 | 31.12 | 39.63 | 78.11 | 47.21 | 60.47 | 78.35 | 38.26 | 48.75 | 78.46 | 45.13 | 57.53 | 78.28 | 50.68 | 64.76 |
| +CausalCode | 78.52 | **18.17** | **23.12** | **79.18** | **19.57** | **21.79** | **78.92** | **19.85** | **25.16** | 78.92 | **34.42** | **39.61** | **79.18** | **25.40** | **29.78** |

the most effective approach for enhancing model robustness among all tested enhancements. (iv) The robustness enhancement effect is stronger for dead code insertion type attacks compared to identifier type attacks. CausalCode can reduce the ASR by more than two-thirds for dead code insertion attacks. For defect detection, we observe similar phenomena. Our method is the most effective among all evaluated approaches in reducing the ASR. This suggests that learning invariant features helps enhance model robustness by minimizing the learned spurious features.

**Experimental results with adversarial attacks on LLMs**. We evaluate our approach on the state-of-the-art code LLM, StarCoder2, revealing its susceptibility to adversarial attacks. For defect detection, StarCoder's accuracy drops from 77.60% to 3.80% under the strongest attack (CARROT-I), with performance declines between 45.70% and 73.80% across various attacks. Notably, CausalCode mitigates these vulnerabilities, significantly reducing performance degradation, with a 39.38 percentage point improvement against CARROT-I. Interestingly, we observe that these LLMs exhibit strong natural robustness against adversarial attacks in the context of function classification, with a performance degradation limited to only 10%. This phenomenon can be attributed to two factors. First, the function classification dataset (POJ-104) is included in the CodeXGLUE benchmark, which was explicitly incorporated into these models' pre-training data. This direct exposure in the pre-training process enables a resulting model to develop particularly robust representations for this task. Second, the pre-training strategy uses the Fill-in-the-Middle (FIM) objectives, which inherently enhance robustness against local perturbations to known data patterns. However, this observation reinforces the importance of our approach, especially in settings where models encounter previously unseen tasks or data patterns, because in these settings the models remain susceptible to adversarial attacks.

Insight 1. *CausalCode outperforms the state-of-the-art methods in enhancing model performance (i.e., effectiveness) and robustness in code classification tasks.*

### 5.3 Visualization for Code Representation Spaces (RQ2)

We validate CausalCode's ability in enhancing model robustness via the code representation spaces learned by different models. For this purpose, we use the POJ-104 dataset because it allows us to inspect the similarity between semantically-equivalent programs. This dataset comprises 104 classes, each containing 500 semantically equivalent programs in different implementations. Since the semantics of all programs under the same problem are the same, the code representations (i.e., vectors) of the programs with equal semantics should be closer to each other than those programs with different semantics. We randomly select 5 classes, with 100 adversarial examples in each class that can evade CodeBERT. The vector marked with the "[CLS]" token serves as the representative of the entire sequence in models like BERT for classification tasks. We apply the T-SNE technique [40] to reduce the vectors to a two-dimensional space for a better visualization effect.

Fig. 7(a) shows that there are no clear separations between examples (i.e., data points) of different classes in ContraBERT, while examples of the same class do not appear in the proximity of each other. In comparison, both adversarial training and CausalCode can cluster examples in the same class together to some extent. Among the two adversarial learning methods, Figs. 7(b) and 7(c) show that ALERT-T, which retrains the model with a large number of adversarial examples, is more effective in distancing classes and clustering examples in the same class than CARROT-T. Fig. 7(d) shows that

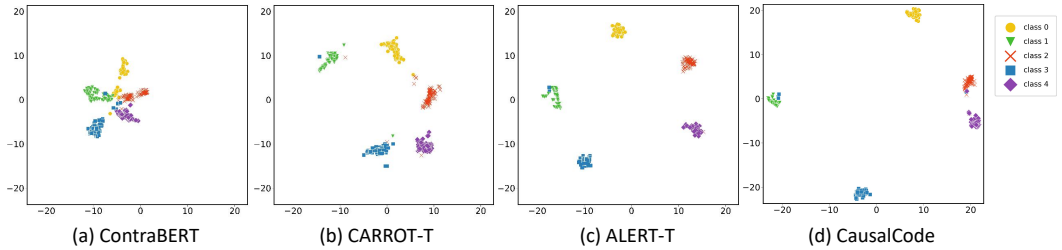



Fig. 7. Visualizing adversarial examples generated by perturbing programs in 5 classes randomly selected from the POJ-104 dataset, where each class has 100 programs and colors represent classes. A greater spatial distance between classes is preferable.

the CausalCode-enhanced model makes examples in the same class cluster tightly together, with clear inter-cluster separations. We do not include CausalVul in this comparison because it requires adding k additional perturbed samples for each input to compute an average, which does not directly reflect its ability to learn representations. This means that CausalCode outperforms ContraBERT and adversarial training in clustering semantically equivalent examples, by capturing the essence of programs. Moreover, by employing K-means [1] for calculating distortion distances between selected examples, we find that CausalCode's distortion distance is the lowest. The distances of ContraBERT, CARROT-T, ALERT-T, and CausalCode are 0.552, 0.531, 0.380, and 0.285, respectively. This indicates the good clustering effect of CausalCode.

Since semantics-preserving adversarial examples do not affect causal features, a robust model should be able to learn causal features such that the representation of adversarial examples should be very close to, if not exactly the same as, that of the legitimate examples from which the adversarial ones are generated. To demonstrate the model learns invariant representations, we conduct experiments to find the original examples corresponding to the feature vectors of given adversarial examples. We rank the original examples based on their similarity to the given adversarial examples, by calculating the proportion of original examples present in the top-ranking nearest neighbor and the top ten nearest neighbors, respectively. To establish baselines, we use the adversarial learning method CARROT-T, the data augmentation method ALERT-T, and the contrastive learning method ContraBERT to conduct experiments on POJ-104 dataset.

Table 4. Comparison in learning invariant representations

| Model | Overlap Rate Top-1(%) | Overlap Rate Top-10(%) | Distance |
|---|---|---|---|
| CodeBERT | 31.81 | 56.36 | 14.41 |
| +ContraBERT | 46.60 | 66.99 | 12.53 |
| +CARROT-T | 40.75 | 63.92 | 11.74 |
| +ALERT-T | 64.32 | 91.47 | 7.56 |
| +CausalCode | **93.32** | **98.40** | **0.94** |

We use the models that are learned via the methods described in Table 4 to process both an original example $x_i$ and an adversarial example $x_i'$ derived from $x_i$, obtaining a learned feature vector for each example. The *top-1-overlap rate* measures the fraction of the original examples that are the nearest neighbors to their respective adversarial examples in the feature space. Similarly, the *top-10-overlap rate* measures the fraction of the original examples that are among the top 10 closest neighbors of their respective adversarial examples. A higher overlap rate indicates a greater ability to learn invariant representations because a higher overlap rate means a model can maintain a high similarity between semantically equivalent examples in the feature space despite adversarial perturbations. This means that such a model learns causal features that are robust against adversarial attacks.

Table 4 summarizes the results. We observe that with respect to the POJ-104 dataset, CausalCode-enhanced models achieve a top-1-overlap rate of 93.32%, which is 45.74% higher than the other methods on average. "Distance" column shows the average distance in the feature space between the original examples and their adversarial examples with respect to the five methods. We observe that CausalCode learns features from the original examples and their adversarial examples with a smaller disparity than other methods. This means that CausalCode effectively accomplishes its design objective: examples with similar semantics are close to each other in the feature space and thus enhances classifier robustness against adversarial examples.

Insight 2. *CausalCode enables DL models to learn improved code representations such that in the feature space, examples with the same causal feature are clustered together.*

### 5.4 Impact of CausalCode Elements on Model Robustness (RQ3)

To demonstrate the contribution of elements in improving the model's robustness, we compare three key elements in CausalCode: intervention examples generation, regularization strategy, and dataset size. For intervention example generation, we contrast our approach with two alternative strategies (see details in Section 3): a *random perturbation* method, which randomly selects samples from all examples generated through random perturbations, and an *adversarial* method, which specifically focuses on adversarial examples. Regarding regularization strategy, we analyze the impact of removing the regularizer from CausalCode, training solely with the expanded samples selected by CausalCode's strategy. Additionally, we compare this approach with a contrastive learning loss. For dataset size, we augment 2-4 training sets, surpassing other models that are limited to expanding just one dataset.

Table 5. Impact of elements on Acc and ASR in CausalCode framework (metrics unit: %)

| Element | Model | MHM | | | CARROT | |
|---|---|---|---|---|---|---|
| | | Acc | $\Delta_{drop}$ | ASR | $\Delta_{drop}$ | ASR |
| CodeBERT | Vanilla | 77.48 | 47.48 | 61.28 | 61.28 | 79.09 |
| Intervention Example Generation | +Causal examples | 78.08 | **23.62** | **30.25** | **26.36** | **33.76** |
| | +Random examples | **78.39** | 41.46 | 52.89 | 54.00 | 68.89 |
| | +Adversarial examples | 78.21 | 41.75 | 53.38 | 45.05 | 57.60 |
| Regularization Strategy | +Causal loss | 78.08 | **23.62** | **30.25** | **26.36** | **33.76** |
| | +Contrastive loss | 78.05 | 44.60 | 57.14 | 57.17 | 73.25 |
| | +Classification loss | **78.54** | 37.24 | 47.42 | 46.51 | 59.22 |
| Dataset Size | +1 augmentation | 78.08 | 23.62 | 30.25 | 26.36 | 33.76 |
| | +2 augmentation | 78.37 | 21.60 | 27.56 | 31.35 | 30.23 |
| | +3 augmentation | **78.41** | **20.26** | **25.84** | **30.03** | **26.98** |
| | +4 augmentation | 78.29 | 21.51 | 27.47 | 30.14 | 28.19 |

Table 5 summarizes the experimental results. We observe three key findings: (i) The random selection method leads to an average decrease in the Attack Success Rate (ASR) of 9.30%, whereas the adversarial method achieves an average reduction of 16.70%. Notably, the CausalCode strategy outperforms both, with a significant reduction in ASR by 38.18% on average. (ii) Absence of the causal learning-derived regularizer results in a marginal improvement in accuracy by 0.46%. However, this comes at the cost of increased vulnerability to adversarial attacks, evidenced by a 21.32% increase in ASR compared to the Causal loss regularizer. The contrastive learning regularizer reduces the ASR, but 33.19% less effectively than the Causal loss regularizer. (iii) Increasing the size of expanded datasets improves the classifier's accuracy and reduces the ASR, but this improvement has limitations.

Insight 3. *Qualitatively speaking, CausalCode can enhance the robustness of DL models because it uses* intervention examples *and* regularization training. *Quantitatively speaking, using more intervention examples leads to a higher robustness.*

### 5.5 Extending to Code Generation Tasks (RQ4)

**Performance comparison of generation tasks**. Table 6 summarizes the effectiveness of CodeBERT and GraphCodeBERT in two generation tasks. The performance improvements attributed to CausalCode surpass those of the baseline. Specifically, for the code translation task, CausalCode achieves an average improvement of 1.18% in CodeBLEU, and for the code refinement task, it achieves an average 0.43% increase in CodeBLEU. In contrast, the performance enhancements from the baseline are negligible.

**Experimental results with adversarial attacks on generation tasks**. As the adversarial samples generated by the CodeAttack method do not guarantee perfect semantic equivalence or syntactic correctness, we exclude samples with divergent semantic and syntactic errors. Following the approach outlined in Li et al. [14], we select the top five adversarial samples generated by CodeAttack to construct an adversarial testing set.

Table 6. Adversarial results on generation tasks for CodeBERT and GraphCodeBERT models, enhanced with four different methods (metrics unit: %)

| Model | Code Translation Java →C# | | Code Refinement Java →Java | |
|---|---|---|---|---|
| | **CodeBLEU** | $\Delta_{drop}$ | **CodeBLEU** | $\Delta_{drop}$ |
| CodeBERT | 82.36 | 16.26 | 87.88 | 6.13 |
| +ContraBERT | 82.51 | 16.76 | 87.85 | 5.96 |
| +CARROT-T | 81.88 | 15.96 | 87.9 | 6.22 |
| +ALERT-T | 82.92 | 15.10 | 87.89 | 5.33 |
| +CausalCode | **83.62** | **13.22** | **88.33** | **5.13** |
| GraphCodeBERT | 82.57 | 16.47 | 87.95 | 10.64 |
| +ContraBERT | 82.52 | 17.62 | 88.05 | 10.75 |
| +CARROT-T | 82.81 | 16.27 | 87.89 | 10.71 |
| +ALERT-T | 82.96 | 16.79 | 87.82 | 11.03 |
| +CausalCode | **83.67** | **16.24** | **88.36** | **10.14** |

Table 6 presents the experimental results. We make several observations: (i) ContraBERT not only fails to reduce the $\Delta_{drop}$ but becomes more vulnerable under CodeAttack. (ii) In code translation and code refinement tasks, the adversarial learning method CARROT-T achieves only a modest reduction in $\Delta_{drop}$ of 0.50% and 0.14% on average, respectively. ALERT-T demonstrates greater efficacy, reducing the $\Delta_{drop}$ by 0.85% and 0.42% on average for these tasks. (iii) CausalCode exhibits the most substantial improvement compared to the baseline, reducing the $\Delta_{drop}$ by 1.63% and 0.76% on average for code translation and code refinement tasks, respectively.

Insight 4. *CausalCode outperforms the state-of-the-art methods in enhancing model performance (i.e., effectiveness) and robustness in code generation tasks.*

## 6 Related Work

**Prior studies on source code models and robustness.** Source code classification includes defect or vulnerability detection [7, 17, 55], code function classification [51, 53], and clone detection [44, 45]. Code generation includes code translation [32], code repair [13], and code summarization [11]. The preceding models achieve remarkable performance (e.g., accuracy), but are susceptible to adversarial attacks [4, 10, 51, 53]. This problem has motivated two major countermeasure approaches: *adversarial training* and *contrastive learning*. (i) The *adversarial training* approach aims to incorporate adversarial examples into the training set, often by perturbing non-adversarial examples into adversarial ones [15, 16, 37, 46, 47, 52]. This approach manifests the idea of *data augmentation*, which increases the diversity of the training data by incorporating examples that may or may not be adversarial [3, 15, 17, 42]. This method leads to models that cannot eliminate spurious features, remaining susceptible to adversarial attacks. (ii) The *contrastive learning* approach aims to minimize the distance between augmented examples of the same example while maximizing the

distance between the examples augmented from different examples. This approach can employ an unsupervised contrastive pre-training task to pre-train a neural network [9, 19]. This approach consumes a lot of resources in retraining large models while still achieving a limited capability against adversarial attacks.

The preceding limitations highlight the importance to seek new approaches, making a case for the causal learning approach proposed in this study. Considering the causal relationships between a model's input and output (i.e., predictions) allows models to learn causal features to enhance robustness against adversarial attacks. Thus, CausalCode initiates a new approach by harnessing causal analysis and causal learning. The causal learning we propose is different from existing contrastive learning because it uses distinct data generation strategies and employs a new form of regularization that is different from the one used in contrastive learning [9].

**Prior studies on causal learning.** Causal learning can enhance the interpretability and robustness of DL models, by capturing the cause-effect relationships hidden in data. Notable results include Pearl's seminal framework for causal inference [27] and recent advancements [34, 36]. These studies lay a foundation for applying causal principles to improve machine learning reliability and generalizability, as demonstrated in image recognition and natural language processing [5, 25]. To our knowledge, the only study that introduces causal learning to source code models is [30], which uses weighted average to cope with spurious features and perturbation attacks. However, this approach achieves limited success in coping with adversarial attacks because it cannot learn invariant representations and thus cannot adequately eliminate spurious features. Whereas, our causal learning technique, which is built upon [8, 24], uses the do-operator to eliminate spurious correlations between code features and model predictions, while learning invariant representations despite the presence of spurious features.

## 7 Limitations and Future Work

This study has several limitations, which need to be addressed in future studies. (i) Our selection of interventions is based on multiple iterations of random sampling in the direction of gradient descent. What would be the other sampling techniques? (ii) Defending against different kinds of adversarial attacks remains a challenge [29, 51]. (iii) It is intuitive that causal learning can improve model interpretability. It is interesting to explore how CausalCode may improve model robustness and interpretability at the same time.

## 8 Conclusion

We have proposed CausalCode, a novel framework for enhancing the robustness of DL-based source code models. By leveraging the causal graph and intervention resistant to adversarial manipulations, CausalCode can distinguish causal features from spurious features. Experimental evaluations with state-of-the-art DL-based classification and generation models show that CausalCode can significantly improve the models' robustness against adversarial attacks. The limitations discussed in Section 7 offer interesting problems for future research.

## Acknowledgements

We thank the anonymous reviewers for their comments, which guided us in improving the paper. The authors affiliated with Huazhong University of Science and Technology were supported by the National Natural Science Foundation of China under Grant No. 62272187. Shouhuai Xu's research was not sponsored / supported by any funding agency. Any opinions, findings, conclusions, or recommendations expressed in this work are those of the authors and do not reflect the views of the funding agencies in any sense.

## References


[1] Mohiuddin Ahmed, Raihan Seraj, and Syed Mohammed Shamsul Islam. 2020. The k-means algorithm: A comprehensive survey and performance evaluation. *Electronics* 9, 8 (2020), 1295. https://doi.org/10.3390/electronics9081295

[2] Seungtaek Choi, Myeongho Jeong, Hojae Han, and Seung-won Hwang. 2022. C2L: Causally Contrastive Learning for Robust Text Classification. In *Proceedings of the 36th AAAI Conference on Artificial Intelligence, Virtual Event*, Vol. 36. 10526–10534. https://doi.org/10.1609/AAAI.V36I10.21296

[3] Rhys Compton, Eibe Frank, Panos Patros, and Abigail Koay. 2020. Embedding Java Classes with code2vec: Improvements from Variable Obfuscation. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR), Seoul, Republic of Korea*. 243–253. https://doi.org/10.1145/3379597.3387445

[4] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Proceedings of the Findings of the Association for Computational Linguistics: (EMNLP), Online Event*. 1536–1547. https://doi.org/10.18653/V1/2020.FINDINGS-EMNLP.139

[5] Yash Goyal, Ziyan Wu, Jan Ernst, Dhruv Batra, Devi Parikh, and Stefan Lee. 2019. Counterfactual Visual Explanations. In *Proceedings of the 36th International Conference on Machine Learning (ICML), Long Beach, California, USA*. 2376–2384. https://doi.org/10.48550/arXiv.1904.07451

[6] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin B. Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCodeBERT: Pre-Training Code Representations with Data Flow. In *Proceedings of the 9th International Conference on Learning Representations (ICLR), Virtual Event, Austria*. https://doi.org/10.48550/arXiv.2009.08366

[7] Jordan Henkel, Goutham Ramakrishnan, Zi Wang, Aws Albarghouthi, Somesh Jha, and Thomas W. Reps. 2022. Semantic Robustness of Models of Source Code. In *Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), Honolulu, HI, USA*. 526–537. https://doi.org/10.1109/saner53432.2022.00070

[8] Maximilian Ilse, Jakub M. Tomczak, and Patrick Forré. 2021. Selecting Data Augmentation for Simulating Interventions. In *Proceedings of the 38th International Conference on Machine Learning (ICML), 18-24 July 2021, Virtual Event (Proceedings of Machine Learning Research, Vol. 139)*. PMLR, 4555–4562. https://doi.org/10.48550/arXiv.2005.01856

[9] Paras Jain, Ajay Jain, Tianjun Zhang, Pieter Abbeel, Joseph E. Gonzalez, and Ion Stoica. 2021. Contrastive Code Representation Learning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), Virtual Event*. 5954–5971. https://doi.org/10.18653/v1/2021.emnlp-main.482

[10] Akshita Jha and Chandan K. Reddy. 2023. CodeAttack: Code-Based Adversarial Attacks for Pre-Trained Programming Language Models. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI), Washington, DC, USA*. 14892–14900. https://doi.org/10.1609/AAAI.V37I12.26739

[11] Jia Li, Yongmin Li, Ge Li, Xing Hu, Xin Xia, and Zhi Jin. 2021. EditSum: A Retrieve-and-Edit Framework for Source Code Summarization. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), Melbourne, Australia*. 155–166. https://doi.org/10.48550/ARXIV.2308.13775

[12] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy V, Jason T. Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. StarCoder: may the source be with you! *Trans. Mach. Learn. Res.* 2023 (2023). https://doi.org/10.48550/arXiv.2305.06161

[13] Yi Li, Shaohua Wang, and Tien N. Nguyen. 2020. DLFix: Context-based code transformation learning for automated program repair. In *Proceedings of the 42nd International Conference on Software Engineering (ICSE), Seoul, South Korea*. 602–614. https://doi.org/10.1145/3377811.3380345

[14] Yiyang Li, Hongqiu Wu, and Hai Zhao. 2022. Semantic-Preserving Adversarial Code Comprehension. In *Proceedings of the 29th International Conference on Computational Linguistics (COLING), Gyeongju, Republic of Korea*. 3017–3028. https://doi.org/10.48550/arXiv.2209.05130

[15] Zhen Li, Guenevere (Qian) Chen, Chen Chen, Yayi Zou, and Shouhuai Xu. 2022. RoPGen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation. In *Proceedings of the 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA*. 1906–1918. https://doi.org/10.1145/3510003.3510181

[16] Zhen Li, Xiang Huang, Yangrui Li, and Guenevere Chen. 2023. A Comparative Study of Adversarial Training Methods for Neural Models of Source Code. *Future Generation Computer Systems* 142 (2023), 165–181. https://doi.org/10.1016/j.future.2022.12.030

[17] Zhen Li, Deqing Zou, Shouhuai Xu, Hai Jin, Yawei Zhu, and Zhaoxuan Chen. 2022. SySeVR: A Framework for Using Deep Learning to Detect Software Vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* 19, 4 (2022), 2244–2258. https://doi.org/10.1109/TDSC.2021.3051525

[18] Shangqing Liu, Yu Chen, Xiaofei Xie, Jing Kai Siow, and Yang Liu. 2021. Retrieval-Augmented Generation for Code Summarization via Hybrid GNN. In *Proceedings of the 9th International Conference on Learning Representations (ICLR), Virtual Event, Austria*. https://doi.org/10.48550/arXiv.2006.05405

[19] Shangqing Liu, Bozhi Wu, Xiaofei Xie, Guozhu Meng, and Yang Liu. 2023. ContraBERT: Enhancing Code Pre-Trained Models via Contrastive Learning. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE), Melbourne, Australia*. 2476–2487. https://doi.org/10.1109/ICSE48619.2023.00207

[20] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2024. StarCoder 2 and The Stack v2: The Next Generation. https://doi.org/10.48550/arXiv.2402.19173 arXiv:2402.19173

[21] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, Joaquin Vanschoren and Sai-Kit Yeung (Eds.). https://doi.org/10.48550/arXiv.2102.04664

[22] Fangrui Lv, Jian Liang, Shuang Li, Bin Zang, Chi Harold Liu, Ziteng Wang, and Di Liu. 2022. Causality Inspired Representation Learning for Domain Generalization. In *Proceedings of the 2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, USA*. 8036–8046. https://doi.org/10.1109/cvpr52688.2022.00788

[23] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. 2018. Towards Deep Learning Models Resistant to Adversarial Attacks. In *Proceedings of the 6th International Conference on Learning Representations (ICLR), Vancouver, BC, Canada*. https://doi.org/10.48550/arXiv.1706.06083

[24] Divyat Mahajan, Shruti Tople, and Amit Sharma. 2021. Domain Generalization Using Causal Matching. In *Proceedings of the 38th International Conference on Machine Learning (ICML), Virtual Event*. 7313–7324. https://doi.org/10.48550/arXiv.2006.07500

[25] Chengzhi Mao, Augustine Cha, Amogh Gupta, Hao Wang, Junfeng Yang, and Carl Vondrick. 2021. Generative Interventions for Causal Learning. In *Proceedings of the 2021 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Virtual Event*. 3947–3956. https://doi.org/10.1109/cvpr46437.2021.00394

[26] Lili Mou, Ge Li, Lu Zhang, Tao Wang, and Zhi Jin. 2016. Convolutional Neural Networks over Tree Structures for Programming Language Processing. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence, Phoenix, Arizona, USA*. 1287–1293. https://doi.org/10.1609/aaai.v30i1.10139

[27] Judea Pearl. 2009. *Causality.* Cambridge University Press. https://doi.org/10.1111/j.1475-6773.2011.01243.x

[28] Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. 2017. *Elements of causal inference: foundations and learning algorithms.* The MIT Press.

[29] Erwin Quiring, Alwin Maier, and Konrad Rieck. 2019. Misleading Authorship Attribution of Source Code Using Adversarial Learning. In *Proceedings of the 28th USENIX Security Symposium (USENIX Security), Santa Clara, CA, USA*. 479–496.

[30] Md Mahbubur Rahman, Ira Ceka, Chengzhi Mao, Saikat Chakraborty, Baishakhi Ray, and Wei Le. 2024. Towards Causal Deep Learning for Vulnerability Detection. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–11. https://doi.org/10.1145/3597503.3639170

[31] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: A Method for Automatic Evaluation of Code Synthesis. *CoRR* abs/2009.10297 (2020). https://doi.org/10.48550/arXiv.2009.10297 arXiv:2009.10297

[32] Baptiste Roziere, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised Translation of Programming Languages. In *Proceedings of the Annual Conference on Neural Information Processing Systems, Virtual*

*Event*. 20601–20611. https://doi.org/10.48550/arXiv.2006.03511

[33] Pedro Sanchez, Jeremy P. Voisey, Tian Xia, Hannah I. Watson, Alison Q. O'Neil, and Sotirios A. Tsaftaris. 2022. Causal Machine Learning for Healthcare and Precision Medicine. *CoRR* abs/2205.11402 (2022). https://doi.org/10.1098/rsos.220638 arXiv:2205.11402

[34] Bernhard Schölkopf. 2022. Causality for Machine Learning. In *Probabilistic and Causal Inference: The Works of Judea Pearl*. Vol. 36. ACM, 765–804. https://doi.org/10.1145/3501714.3501755

[35] Bernhard Schölkopf, Francesco Locatello, Stefan Bauer, Nan Rosemary Ke, Nal Kalchbrenner, Anirudh Goyal, and Yoshua Bengio. 2021. Toward Causal Representation Learning. *Proc. IEEE* 109, 5 (2021), 612–634. https://doi.org/10.1109/jproc.2021.3058954

[36] Bernhard Schölkopf, Francesco Locatello, Stefan Bauer, Nan Rosemary Ke, Nal Kalchbrenner, Anirudh Goyal, and Yoshua Bengio. 2021. Toward Causal Representation Learning. *Proc. IEEE* 109, 5 (2021), 612–634. https://doi.org/10.1109/jproc.2021.3058954

[37] Jacob M. Springer, Bryn Marie Reinstadler, and Una-May O'Reilly. 2021. STRATA: Simple, Gradient-Free Attacks for Models of Code. *CoRR* abs/2009.13562 (2021). arXiv:2009.13562

[38] Shashank Srikant, Sijia Liu, Tamara Mitrovska, Shiyu Chang, Quanfu Fan, Gaoyuan Zhang, and Una-May O'Reilly. 2021. Generating Adversarial Computer Programs Using Optimized Obfuscations. In *Proceedings of the 9th International Conference on Learning Representations (ICLR), Virtual Event, Austria*. https://doi.org/10.48550/arXiv.2103.11882

[39] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 4 (2019), 1–29. https://doi.org/10.1145/3340544

[40] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of Machine Learning Research* 9, 11 (2008).

[41] Morteza Verdi, Ashkan Sami, Jafar Akhondali, Foutse Khomh, Gias Uddin, and Alireza Karami Motlagh. 2022. An Empirical Study of C++ Vulnerabilities in Crowd-Sourced Code Examples. *IEEE Trans. Software Eng.* 48, 5 (2022), 1497–1514. https://doi.org/10.1109/tse.2020.3023664

[42] Deze Wang, Zhouyang Jia, Shanshan Li, Yue Yu, Yun Xiong, Wei Dong, and Xiangke Liao. 2022. Bridging Pre-trained Models and Downstream Tasks for Source Code Understanding. In *Proceedings of the 44th International Conference on Software Engineering (ICSE), Pittsburgh, PA, USA*. 287–298. https://doi.org/10.1145/3510003.3510062

[43] Wenhan Wang, Ge Li, Bo Ma, Xin Xia, and Zhi Jin. 2020. Detecting Code Clones with Graph Neural Network and Flow-Augmented Abstract Syntax Tree. In *Proceedings of the 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), London, ON, Canada*. 261–271. https://doi.org/10.1109/saner48275.2020.9054857

[44] Yueming Wu, Shihan Dou, Deqing Zou, Wei Yang, Weizhong Qiang, and Hai Jin. 2022. Contrastive Learning for Robust Android Malware Familial Classification. *IEEE Transactions on Dependable and Secure Computing* (2022), 1–14. https://doi.org/10.1109/tdsc.2022.3219082

[45] Yueming Wu, Deqing Zou, Shihan Dou, Siru Yang, Wei Yang, Feng Cheng, Hong Liang, and Hai Jin. 2021. SCDetector: Software Functional Clone Detection Based on Semantic Tokens Analysis. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), New York, NY, USA*. 821–833. https://doi.org/10.1145/3324884.3416562

[46] Zhou Yang, Jieke Shi, Junda He, and David Lo. 2022. Natural Attack for Pre-Trained Models of Code. In *Proceedings of the 44th International Conference on Software Engineering (ICSE), New York, NY, USA*. 1482–1493. https://doi.org/10.1145/3510003.3510146

[47] Noam Yefet, Uri Alon, and Eran Yahav. 2020. Adversarial examples for models of code. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 162:1–162:30. https://doi.org/10.1145/3428230

[48] Shiwen Yu, Ting Wang, and Ji Wang. 2022. Data Augmentation by Program Transformation. *Journal of Systems and Software* 190 (2022), 111304. https://doi.org/10.1016/j.jss.2022.111304

[49] Xiaoyong Yuan, Pan He, Qile Zhu, and Xiaolin Li. 2019. Adversarial Examples: Attacks and Defenses for Deep Learning. *IEEE Transactions on Neural Networks and Learning Systems* 30, 9 (2019), 2805–2824. https://doi.org/10.1109/tnnls.2018.2886017

[50] Cheng Zhang, Kun Zhang, and Yingzhen Li. 2020. A Causal View on Robustness of Neural Networks. In *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS), Virtual Event*, Vol. 33. 289–301. https://doi.org/10.48550/arXiv.2005.01095

[51] Huangzhao Zhang, Zhiyi Fu, Ge Li, Lei Ma, Zhehao Zhao, Hua'an Yang, Yizhe Sun, Yang Liu, and Zhi Jin. 2022. Towards Robustness of Deep Program Processing Models—Detection, Estimation, and Enhancement. *ACM Transactions on Software Engineering and Methodology* 31, 3 (2022), 50:1–50:40. https://doi.org/10.1145/3511887

[52] Huangzhao Zhang, Zhuo Li, Ge Li, Lei Ma, Yang Liu, and Zhi Jin. 2020. Generating Adversarial Examples for Holding Robustness of Source Code Processing Models. In *Proceedings of the 2020 AAAI Conference on Artificial Intelligence, New York, NY, USA*. 1169–1176. https://doi.org/10.1609/aaai.v34i01.5469

[53] Jian Zhang, Xu Wang, Hongyu Zhang, Hailong Sun, Kaixuan Wang, and Xudong Liu. 2019. A novel neural source code representation based on abstract syntax tree. In *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019*, Joanne M. Atlee, Tevfik Bultan, and Jon Whittle (Eds.). IEEE / ACM, 783–794. https://doi.org/10.1109/ICSE.2019.00086

[54] Yonggang Zhang, Mingming Gong, Tongliang Liu, Gang Niu, Xinmei Tian, Bo Han, Bernhard Schölkopf, and Kun Zhang. 2022. CausalAdv: Adversarial Robustness through the Lens of Causality. arXiv:2106.06196

[55] Deqing Zou, Sujuan Wang, Shouhuai Xu, Zhen Li, and Hai Jin. 2021. $\mu$VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection. *IEEE Transactions on Dependable and Secure Computing* 18, 5 (2021), 2224–2236. https://doi.org/10.1109/TDSC.2019.2942930

Received 2025-02-26; accepted 2025-04-01